BEST Mumbai Embedded Ticket Kiosk

A Multilingual, Click-as-Touch Ticket Booking Kiosk Prototype

This is submitted as Final Project Report of the Course

Human Computer Interaction

of

Third Year

of four year graduation course for

Bachelor of Technology in Information Technology

(Semester V)

Under the guidance of

Dr. V K Sambhe

CE & IT Department, VJTI, Mumbai

Submitted by

Mr. Awwab Wadekar	241080077
Mr. Aditya Shimoga	241080067

Veermata Jijabai Technological Institute,

H.R. Mahajani Road

Matunga, Mumbai 19

Veermata Jijabai Technological Institute, Matunga, Mumbai - 400019

CERTIFICATE

This is to certify that the project entitled “BEST Mumbai Embedded Ticket Kiosk” submitted by Mr. Awwab Wadekar (Reg. No. 241080077) and Mr. Aditya Shimoga (Reg. No. 241080067) in the partial fulfillment of the requirements for the subject Human Computer Interaction of Third Year (Semester V) of four year course for Bachelor of Technology Degree in Information Technology at Veermata Jijabai Technological Institute, Matunga, Mumbai - 400019 is an authentic work carried out under the guidance of Dr. V K Sambhe.

To the best of my knowledge, the matter embodied in the project has not been submitted to any other University/Institute for the award of any Degree or Diploma course.

Date: October 20, 2026
Place: Mumbai

Dr. V K Sambhe
Department of CE & IT
Veermata Jijabai Technological Institute, Mumbai

STATEMENT BY CANDIDATE

We wish to state that work embodied in this special topic titled “Human Computer Interaction” forms our own contribution to the work carried out under the guidance of Dr. V K Sambhe at the ‘Veermata Jijabai Technological Institute’, Matunga, Mumbai – 19. This work has not been submitted for any other Degree or Diploma of any University / Institute. Wherever references have been made to previous works of others, it has been clearly indicated.

Mr. Awwab Wadekar (241080077)
Mr. Aditya Shimoga (241080067)

Third Year, B.Tech IT
Department of CE & IT
Veermata Jijabai Technological Institute, Mumbai

Awwab Wadekar

Aditya Shimoga

Abstract

The BEST Mumbai Embedded Ticket Kiosk is a graphical, click-as-touch ticket-booking kiosk prototype built for Brihanmumbai Electric Supply and Transport (BEST) Mumbai's bus network. The application is written in embedded C for its route, booking, localization, payment, and printer logic, with SDL2 used only to emulate a touch-screen display during development, so that the same business-rule code can later be ported to a real embedded display/touch controller without being rewritten.

Users interact with the kiosk entirely through taps (mouse clicks standing in for touch): picking a bus route from a searchable, scrollable list or an offline satellite map of Mumbai; selecting a start and destination stop by tapping either the map or a list, which stay in sync with each other; choosing a payment method (cash, card, or UPI); and receiving a time-limited, scannable QR-coded ticket that can be saved or printed as a PDF. A dedicated Timetable screen shows the real departure schedule for the selected route, and a Help screen surfaces the helpline number, an emergency number, and data attributions without derailing an in-progress booking.

The catalogue behind the kiosk is not mock data: it is the real BEST route and stop network (hundreds of routes, thousands of stops) imported once from the community-maintained croyla/mumbai-gtfs feed and baked into fixed-size C arrays at build time, so the firmware path needs no parser or dynamic memory allocation regardless of catalogue size. The UI is available in four languages -- English, Marathi, Hindi, and Gujarati -- each with its own bundled font, and an on-screen Latin keypad lets a rider who already knows a route number or stop name jump straight to it instead of scrolling past hundreds of entries.

In terms of future scope, the kiosk has room to grow toward a full embedded deployment: porting the display/touch path to a constrained microcontroller target, wiring the simulated printer-paper sensor to a real GPIO printer driver, consuming GTFS-realtime feeds for live delays instead of a static timetable, and adding contactless/NFC payment. By digitising a presently manual, conductor-issued paper-ticket process, the kiosk aims to bring consistency, multilingual accessibility, and auditable error handling to BEST's everyday ticketing experience.

TABLE OF CONTENTS

Abstract **4**

Introduction **7**

Introduction 7

Problem Statement 7

Idea Behind the Project 7

Objectives 8

Preliminary Design **9**

Present System 9

Limitation of Present System 9

Proposed Idea 9

Significance 10

Proposed Solution **11**

Features 11

Interactive System Design **13**

Usability and Usability Engineering 13

User Centered Design 13

Participatory Design 14

Prototyping 14

Model Based Design **15**

Different Types of Models 15

Keystroke Level Model 15

CMN(GOMS) 16

Fitts Law 17

Hick-Hyman's Law 17

Guidelines in HCI **19**

Shneiderman's eight golden rules 19

Norman's seven principles 19

Norman's model of interaction 20

Nielsen's ten heuristics 21

Task Modeling and Analysis **22**

Hierarchical Task Analysis (HTA) 22

Concur Task Tree (CTT) 22

Evaluation Techniques **25**

Cognitive Walkthrough 25

Heuristic Evaluation 26

Model Based Evaluation 27
Evaluation with 5 goals of Usability Engineering 27
Implementation **28**
Technology Used 28
Results 28
Future Scope 35
Conclusion **37**

Introduction

Introduction

Public bus ticketing in large transit networks such as Brihanmumbai Electric Supply and Transport (BEST) Mumbai has traditionally relied on a conductor manually issuing paper tickets after verbally confirming a rider's destination and collecting cash fare. This process does not scale well to self-service kiosks, does not inherently support multiple languages, and leaves no automatic digital or printed trail beyond the paper ticket itself.

The BEST Mumbai Embedded Ticket Kiosk project builds a self-service, touch-operated alternative: a kiosk application, written primarily in embedded C, that lets a rider independently pick a route, pick where they board and alight, pay by cash, card, or UPI, and walk away with a scannable, time-limited QR ticket -- without needing a conductor, a smartphone app, or literacy in any one specific language.

Because the project is explicitly an embedded-systems exercise as well as an HCI one, the kiosk's business logic (route/stop catalogue, fare calculation, booking validation, translated strings, map projection math) is written as SDL-free, unit-tested C modules, with SDL2 used only to emulate the eventual touch display during development. This keeps the architecture realistic to how a production kiosk running on dedicated hardware would actually be structured, while still letting the interaction design be evaluated on a desktop today.

Problem Statement

Manually issued paper tickets do not scale to self-service kiosks and create several concrete usability gaps for BEST Mumbai riders:

- A conductor-mediated transaction assumes a shared language between rider and conductor; a kiosk serving Mumbai's linguistically diverse ridership cannot make that assumption.
- Fare calculation and destination confirmation are done verbally and from memory, with no visible, re-checkable record before money changes hands.
- There is no structural protection against a kiosk silently failing to produce a ticket (for example, running out of receipt paper) while a rider is still charged.
- Mock or hand-curated route data does not expose the real usability challenges of the actual BEST network, which has hundreds of routes and thousands of stops -- an interface designed only against a handful of invented routes would not actually prove out at real scale.

The kiosk responds to each of these directly: a four-language UI removes the shared-language assumption, an explicit confirm-before-pay flow with a visible fare makes the transaction checkable, booking validation plus a simulated paper-stock sensor make failure states visible and recoverable rather than silent, and the catalogue used throughout is the real, full-size BEST GTFS feed rather than a curated toy subset.

Idea Behind the Project

The idea is to prove out, on a desktop SDL2 emulator today, the exact interaction model a real embedded kiosk would need tomorrow: a rider taps through a strict, closed sequence -- route, then stops, then payment, then ticket -- where every screen offers only touch-sized buttons, a Back action, and immediate visible feedback, never a keyboard, pointer-fine control, or hidden menu a public kiosk audience could not be expected to use.

Rather than inventing a simplified mock bus network, the project deliberately imports BEST's real route/stop catalogue and a real satellite basemap of Mumbai, so that every usability decision -- list pagination, on-screen search, label truncation, map/list redundancy -- is validated against the actual scale and messiness of production data, not a convenient toy dataset.

The same discipline carries into the architecture: input handling, drawing, business rules, and configuration are kept in separate modules, with the booking/catalogue/localization/map-projection logic written to be unit tested independently of any graphics library, so the project reads as a believable first step toward real embedded hardware rather than a purely cosmetic prototype.

Objectives

The main objectives of the BEST Mumbai Embedded Ticket Kiosk project are as follows:

- **Build a touch-only, click-as-touch interaction model:** every control on every screen must be operable with nothing finer than a single tap, with no assumption of a keyboard, stylus, or precise pointer, consistent with a public kiosk audience.
- **Deliver multilingual access as a first-class requirement:** ship a fully hand-translated UI in English, Marathi, Hindi, and Gujarati, each backed by its own bundled font, rather than treating language as an afterthought bolted onto an English-only design.
- **Work against the real BEST network, not mock data:** import the actual BEST route/stop catalogue and a real satellite basemap of Mumbai so the interface is proven at real scale -- hundreds of routes and thousands of stops.
- **Keep the architecture realistic to an embedded target:** separate input/drawing (SDL2) from business logic (route catalogue, booking rules, localization, map projection), so the same unit-tested core can later move to real embedded display/touch hardware.
- **Make failure visible and recoverable, not silent:** validate a booking before money changes hands, and surface a simulated printer-paper-empty condition as a plain-language, translated notice rather than a crash or a silently lost ticket.
- **Apply recognised HCI guidelines traceably:** tie each design decision back to a specific, named rule from Shneiderman's, Norman's, and Nielsen's guidelines rather than asserting usability informally.

Preliminary Design

Present System

BEST Mumbai's present ticketing process for most routes is conductor-mediated: a rider boards, states or is asked their destination, the conductor mentally (or with a handheld ticketing machine) computes the fare, collects cash, and tears off or prints a paper ticket. The entire transaction depends on a conductor being present, understanding the rider's spoken language, and being free to attend to that rider promptly.

Where self-service or unattended ticketing does exist, it is typically built around a fixed, small set of hand-curated routes and stops rather than the full real-world network, which hides exactly the scale and data-quality problems -- long stop names, duplicate route numbers for up/down directions, hundreds of routes -- that a genuinely usable kiosk has to solve.

Limitation of Present System

- **Language dependency:** a conductor-mediated transaction assumes rider and conductor share a language; it offers nothing to a rider who does not speak any language the conductor is comfortable in.
- **No visible pre-payment confirmation:** fare and destination are confirmed verbally and from memory, with no on-screen or printed record the rider can check before paying.
- **No structural error handling:** nothing prevents a rider from being charged while the conductor is unable to actually issue a ticket (for example, no ticket stock left); the failure is handled informally, if at all.
- **Does not scale to self-service:** off-peak services, short-staffed routes, or a desire to reduce queueing all need an unattended option, which the present process has no equivalent of.
- **No usable record for the rider:** a torn paper ticket carries no scannable reference a rider or inspector can independently verify.
- **Prototype systems built on toy data:** any earlier mock-up built from a handful of invented routes cannot expose the real usability pressure of the actual network's size (hundreds of routes, thousands of stops, duplicate route numbers, long stop names).

Proposed Idea

The BEST Mumbai Embedded Ticket Kiosk directly answers the limitations above with a self-service, touch-only application built against the real BEST network:

- Multilingual by construction: a fully hand-translated UI in English, Marathi, Hindi, and Gujarati, each rendered with its own bundled font so text never silently fails to display.
- A visible, checkable transaction: the selected route, start stop, destination stop, and computed fare are all shown together on the payment screen before the rider commits to paying.
- Structural error handling: a booking is validated (a route and two distinct, in-range stops must be chosen) before payment, and a simulated paper-stock sensor surfaces a translated notice instead of silently failing to produce a ticket.
- A digital, scannable record: every ticket carries a QR code encoding a unique ticket reference, and can additionally be saved or printed as a PDF.
- Built against the real network: the real BEST GTFS route/stop catalogue and a real satellite basemap of Mumbai, not a hand-curated subset, so the interface is proven at production scale.

By implementing these ideas, the kiosk aims to bring a self-service option to BEST's ticketing experience without sacrificing the language accessibility, visible confirmation, and error recovery that a conductor-mediated transaction happens to provide informally today.

Significance

The BEST Mumbai Embedded Ticket Kiosk holds significance both as a public-transit usability exercise and as an embedded-systems one. It demonstrates that a touch-only, multilingual, self-service kiosk can be built against a real, large-scale transit network rather than a simplified mock-up, and that doing so surfaces interaction problems (pagination, search, label truncation, duplicate route numbers) that a toy dataset would never reveal. By keeping the business logic SDL-free and unit tested, the project also demonstrates a realistic path from a desktop interaction prototype to an eventual embedded display/touch target, which is directly relevant to how real kiosk hardware for a transit authority such as BEST would actually be developed and maintained.

Proposed Solution

Features

1. Route Selection (Map and Searchable List)

The route screen is the kiosk's entry point. A rider sees a satellite map of Mumbai with every real BEST stop drawn as a small dot, alongside a scrollable sidebar list of route numbers, each drawn in that route's own colour. Tapping a row in the list selects that route and immediately recentres and zooms the map onto it, so the rider's next action (picking stops) always starts from a relevant view rather than a city-wide overview.

Because the real BEST network has close to five hundred routes, the list is deliberately paginated (eight rows visible at a time, with grey/blue scroll arrows that disable themselves at either end) rather than attempting to show everything at once, keeping the screen legible without hiding the fact that the full network is there to browse.

2. Stop Selection with Map and List Redundancy

Once a route is chosen, the Stops screen asks the rider to tap a start stop and then a destination stop. Critically, the map and the sidebar list are two views of the same underlying selection: tapping a stop's dot on the satellite map and tapping the same stop's row in the list do exactly the same thing, and the chosen start stop turns green while the destination turns orange in both views simultaneously.

Tapping an already-selected stop again un-picks it, so a rider can correct a mistake with a single tap instead of navigating all the way back to the route screen and starting over.

3. On-Screen Keypad Search for Routes and Stops

With close to five hundred routes and thousands of stops, scrolling alone does not scale for a rider who already knows exactly what they want. Tapping the search field on either the Route or the Stops screen opens a full-screen on-screen keypad (Latin letters and digits only, matching the English-only source data the real route codes and stop names use), echoing the typed query with a blinking cursor and offering Clear, backspace, and Done controls.

The underlying filter is a case-insensitive substring match shared by both screens, so a rider who knows a route number (for example "213") or part of a stop name (for example "Churchgate") can jump straight to it instead of scrolling past hundreds of unrelated entries.

4. Four-Language Multilingual UI

Every screen is available in English, Marathi, Hindi, and Gujarati, switched with a single tap on a button in the header that is always labelled in the language it will switch to next. All fixed UI text (titles, buttons, error notices, the help screen) is hand-translated, while route and stop names are transliterated from the English GTFS source into Devanagari and Gujarati script.

Each language is rendered with its own bundled font rather than relying on whatever happens to be installed on the host or target device, since a UI that silently falls back to empty boxes for an entire script is, for the rider who cannot read it, as unusable as having no translation at all.

5. Offline Satellite Basemap with Discrete Zoom and Pan

The map is a real Sentinel-2 satellite mosaic of Mumbai, shown entirely offline (the kiosk itself never touches the network once built). Zoom is two discrete "+"/"-" buttons rather than a pinch gesture, and tapping empty map area re-centres the view -- both choices keep navigation reachable with a single tap, consistent with the kiosk's click-as-touch-only input model.

The camera is clamped so it can never pan past the edge of the basemap image, and every real stop in the network is always drawn as a small dot -- the map never renders all routes' coloured lines at once, only the currently selected route's, which keeps the screen legible even though the underlying dataset is the full, real BEST network rather than a curated subset.

6. Bus Timetable Lookup

Once a route is picked, a Timetable button becomes available in the header on both the Route and Stops screens (it stays greyed out until then, since there is nothing to show a schedule for yet). Tapping it shows the selected route's departure times as a fixed grid that needs no scrolling or search, laid out so the entire (already time-sampled) schedule fits on one screen at once.

A Back button returns the rider to exactly the screen the Timetable button was pressed from, so dipping into the schedule never costs the rider their place in the booking flow.

7. Fare Calculation and Payment Method Selection

The Payment screen shows the chosen route, start stop, destination stop, and computed fare together before any money changes hands, then lets the rider pick Cash, Card, or UPI as the payment method with a single tap -- the selected method is highlighted green so the choice is never ambiguous.

Fare is computed from the distance, in stop-index terms, between the chosen start and destination stops, giving a transparent, rechecked number the rider sees on screen rather than one quoted verbally.

8. Scannable QR-Coded Digital Ticket

Completing payment produces a ticket carrying a scannable QR code that encodes a unique ticket reference. The QR is intended for prototype scanning by a ticketing backend or inspector's device; validating it in production would require such a backend, which is out of scope for this kiosk prototype but is a natural next integration point.

9. Save/Print Ticket as PDF

From the Ticket screen, a rider can save the ticket as a high-resolution PDF or send it straight to print, both rendered from the exact same on-screen ticket layout (route, boarding stop, destination, fare, QR code, and a 90-minute validity window) so there is never a mismatch between what is shown on screen and what is produced on paper.

10. Printer-Paper Error Handling

The kiosk simulates a printer-paper-stock sensor. If the booking is otherwise valid but the simulated stock is empty, the kiosk refuses to silently charge the rider: it surfaces a plain-language, translated notice ("Printer needs paper") directly on the Payment screen instead of producing a ticket, letting the rider see and act on the problem rather than discovering it only after paying.

11. Helpline and Dedicated Help Screen

A dedicated Help screen, reachable with one tap from the Route screen without losing an in-progress booking, surfaces the helpline number, an emergency number, and the data/imagery attributions the project is required to carry. Keeping this one tap away, rather than nested in a settings menu, follows the same large-target, nothing-hidden philosophy as every other screen in the kiosk.

Interactive System Design

Usability and Usability Engineering

Usability

'Usability' is the measure of the quality of a user's experience when interacting with a product or system.

Usability is crucial in HCI to ensure that public kiosk interfaces are learnable, efficient, and forgiving for a first-time, unaided user. One concrete example in this project is the layout of the route list and map: a well-designed kiosk screen should make the next required action (pick a route, then pick two stops) obvious without instructions, keep touch targets large and consistently placed, and let a rider recover from a mis-tap with a single further tap. Considerations such as consistent colour meaning (green for a start point, orange for a destination, grey for a disabled control), redundant map/list selection, and visible scroll state all contribute to the kiosk's usability, since a public kiosk cannot assume a rider who has read a manual or received training.

Usability Engineering

'Usability Engineering' is the process of deriving, specifying, measuring, constructing, and evaluating usability features into products and systems. It is essentially a research-and-design-based activity focused on design and implementation processes. Usability engineering in HCI is essential for a kiosk application because it helps optimise task efficiency and error tolerance for a rider under no obligation to persevere through a confusing flow. For example, consider the kiosk's booking flow: by employing usability engineering principles, the design can streamline route-then-stops-then-payment into a strictly sequenced set of screens, minimise the number of required taps, provide constantly visible state (selected route, selected stops, fare), and offer a printer-stock error as a recoverable notice rather than a dead end. Through iterative design and testing, issues such as an ambiguous stop label or an unreachable Back button can be identified and addressed, with the ultimate aim of creating a positive experience that increases completed bookings and reduces abandoned transactions.

User Centered Design

User Centered Design (UCD) is a human-computer interaction (HCI) methodology that gives users' wants, preferences, and behaviour top priority when creating technological systems or user interfaces. In order to design intuitive, effective, and pleasurable goods and services, it focuses on understanding users' goals, tasks, and contexts.

Users are actively involved in the design process in UCD at every stage of the development lifecycle, from preliminary research and analysis to iterative design and evaluation. By doing this, it is made sure that the finished product meets the needs, expectations, and capabilities of the users -- in this project's case, BEST riders who cannot be assumed to share a language, read fine print, or have used a similar kiosk before.

Key principles of user-centered design include:

- **User involvement:** riders of varying language and literacy levels are the design target throughout, rather than a single assumed "typical" user.
- **Empirical measurement:** design decisions (touch target sizes, list pagination, search affordance) are grounded in the real scale of the BEST network rather than guesswork.
- **Iterative design:** the booking flow, map/list redundancy, and error notices are refined through repeated passes rather than a single, fixed first draft.
- **Focus on usability:** usability is a primary objective, aiming to make the kiosk easy to learn, efficient to use, and tolerant of mis-taps.

- **Contextual understanding:** the design accounts for the specific context of use -- a public, unattended, touch-only kiosk -- rather than assuming a desktop mouse-and-keyboard user.

By embracing user-centered design, the project aims to produce a kiosk that improves rider confidence and task success, ultimately resulting in more successful, less frustrating ticket-booking interactions.

Participatory Design

Human-computer interaction (HCI) experts refer to participatory design as a cooperative strategy where users and stakeholders actively participate in the creation of technological systems or interfaces. It tries to include end users early in the design process to make sure their requirements, viewpoints, and expertise are incorporated into the finished product.

In participatory design, users are actively involved in various design processes like ideation, prototype, and assessment. Workshops, interviews, observations, and other interactive techniques are frequently used in the process to acquire user feedback. For a kiosk intended for riders across several languages and literacy levels, this kind of early involvement is especially important for catching problems (an unclear icon, an ambiguous stop name, a colour convention that does not read clearly) that a design team working in isolation would be unlikely to notice.

Participatory Design encourages user pleasure, usability, and effectiveness of the finished product by including consumers directly in the design process. By fostering a sense of ownership among users and revealing user requirements that might not be obvious to designers, it promotes greater acceptance and adoption of the technology.

Overall, Participatory Design encourages a more inclusive and user-centric approach to HCI by recognising the significance of the user's voice and engagement in determining the design of technological systems such as this kiosk.

Prototyping

A prototype is an early, condensed version or representation of a technological system or interface used in human-computer interaction (HCI). It functions as a concrete and interactive model that enables designers, developers, and consumers to see and assess the design ideas and features of a product before it is really implemented.

In the iterative design process, prototyping is essential because it allows designers to get user input, test hypotheses, and make adjustments to the system as a result. By giving stakeholders first-hand experience, it aids in bridging the gap between conceptual ideas and practical implementation. This project's own SDL2 desktop build is itself exactly such a prototype: it stands in for a real embedded touch display, letting the interaction design be tested, screenshotted, and evaluated well before any physical kiosk hardware is built.

The fidelity of a prototype might range from rough sketches or paper-based mock-ups to highly detailed interactive computer simulations. The decision of fidelity is affected by the objectives of the design, the resources at hand, and the stage of the design process. High-fidelity prototypes are closer to the finished product and allow for more realistic user interactions and feedback, whereas low-fidelity prototypes are frequently used in the early phases to explore and communicate design concepts.

Prototypes help designers test design choices, find and fix usability problems, and comprehend user wants and preferences better -- in this project, the SDL2 emulator is a high-fidelity prototype that already runs the exact booking/catalogue/localization logic intended for the final embedded target.

Model Based Design

Different Types of Models

A 'model' in HCI refers to "a representation of the user's interaction behavior under certain assumptions". The representation is typically obtained from extensive empirical studies (collecting and analysing data from end users). The model represents the behaviour of average users, not individuals.

There are two broad categorisations of the models used in HCI:

- **Descriptive/prescriptive models:** some models in HCI are used to explain/describe user behaviour during interaction in qualitative terms. An example is Norman's model of interaction. These models help in formulating (prescribing) guidelines for interface design.
- **Predictive engineering models:** these models can "predict" behaviour of a user in quantitative terms. An example is the GOMS model, which can predict the task completion time of an average user for a given system.

The predictive engineering models used in HCI are of three types:

- **Formal (system) models:** the interactive system (interface and interaction) is represented using 'formal specification' techniques.
- **Cognitive (user) models:** these models capture the user's thought (cognitive) process during interaction, approached via simple models of human information processing, individual models of human factors, or integrated cognitive architectures.
- **Syndetic (hybrid) model:** in this model, both the system (external aspect) and the cognitive activities (internal aspect) are combined and represented using formal specification.

Keystroke Level Model

The KLM model provides a quantitative tool. The model allows a designer to 'predict' the time it takes for an average user to execute a task using an interface and interaction method.

Working of the KLM model:

- In KLM, it is assumed that any decision-making task is composed of a series of 'elementary' cognitive (mental) steps, that are executed in sequence.
- These 'elementary' steps essentially represent low-level cognitive activities, which cannot be decomposed any further.
- The method of breaking down a higher-level cognitive activity into a sequence of elementary steps is simple to understand, provides a good level of accuracy, and enough flexibility to apply in practical design situations.

KLM (Keystroke-Level Model) time for performing two of the kiosk's own features --

Feature 1: Selecting a Route and Two Stops

- Tap a route row: 0.20 seconds (tap/click)
- Point to the desired stop on the map or list: 1.10 seconds (pointing task)
- Tap the start stop: 0.20 seconds (tap/click)
- Point to the destination stop: 1.10 seconds (pointing task)
- Tap the destination stop: 0.20 seconds (tap/click)

- Point to and tap the Continue button: 1.10 + 0.20 seconds

Total time for performing Feature 1: 4.10 seconds

Feature 2: Completing Payment

- Point to a payment method (Cash/Card/UPI): 1.10 seconds (pointing task)
- Tap the chosen payment method: 0.20 seconds (mouse click)
- Tap the "Print ticket" button: 0.20 seconds (mouse click)

Total time for performing Feature 2: 1.50 seconds

CMN(GOMS)

In CMN(GOMS), a hierarchical cognitive (thought) process is assumed, as opposed to the linear thought process of KLM.

CMN(GOMS) allows us to model the task and user actions in terms of four constructs (goals, operators, methods, selection rules).

- **Goals:** represents what the user wants to achieve, at a higher cognitive level. This is a way to structure a task from a cognitive point of view.
- **Operators:** elementary acts that change the user's mental (cognitive) state or task environment. This is similar to the operators encountered in KLM, but here the concept is more general.
- **Methods:** these are sets of goal-operator sequences to accomplish a sub-goal.
- **Selection rules:** sometimes there can be more than one method to accomplish a goal. Selection rules provide a mechanism to decide among the methods in a particular context of interaction.

GOMS Structure for Feature 1: Selecting a Route and Two Stops

Goal:

The goal of the route-and-stops selection feature is to let a rider identify which bus to board and where to board and alight, within the BEST Mumbai Embedded Ticket Kiosk application.

Operators:

- Vision: the rider visually perceives the route list, the map, and the stop dots.
- Cognitive: the rider recalls or recognises their route number and the names/locations of their stops.
- Motor: the rider taps the relevant rows or map dots.

Method:

- Scroll or search the route list to locate the desired route number.
- Tap the route row to select it and move to the Stops screen.
- Tap the start stop, either on the map or in the list.
- Tap the destination stop, either on the map or in the list.
- Tap "Continue" to proceed to payment.

Selection rule:

- If the rider already knows the route number or stop name, they use the on-screen keypad search instead of scrolling.

- If the rider recognises their stop visually, they tap the map; if they recognise it by name, they tap the list -- either method reaches the identical selection state.

GOMS Structure for Feature 2: Completing Payment

Goal:

The goal of the payment feature is to let the rider confirm the fare and pay, receiving a valid ticket in exchange.

Method:

- Read the displayed route, stops, and fare.
- Tap the preferred payment method (Cash, Card, or UPI).
- Tap "Print ticket" to complete the booking.

Selection rule:

- The rider may choose any of the three payment methods interchangeably; the booking logic treats all three identically once selected.

Fitts Law

Fitts' law is a model of human motor performance.

Important thing about Fitts's Law is that it is both a descriptive and a predictive model.

- Descriptive because it provides "throughput", which is a descriptive measure of human motor performance.
- Predictive because it provides a prediction equation (an analytical expression) for the time to acquire a target, given the distance and size of the target.

Fitts's Law in HCI is primarily concerned with predicting the time required to reach a target based on its size and distance. Applying it to the kiosk's own layout, two deliberate choices stand out.

First, the route-row and stop-row buttons are drawn as wide, full-width rectangles (240 pixels wide by roughly 40 pixels tall) rather than small, densely packed list items -- a large effective target area that reduces the precision a rider's tap needs, which matters even more on a real touch panel than with a desktop mouse pointer.

Second, the three payment-method buttons on the Payment screen are drawn as large, widely spaced squares (190 by 80 pixels each, separated by a clear gap) rather than small radio buttons placed close together. By Fitts's Law, both a larger target and a more generously spaced one reduce acquisition time and misclick risk; placing the "Continue" and "Print ticket" buttons centrally, directly below the information a rider just read, similarly keeps the distance the rider's pointer or finger must travel short relative to where their attention already is.

Hick-Hyman’s Law

While Fitts' law relates task performance to motor behaviour, Hick-Hyman's law, popularly used in HCI, tells us the "reaction time" (i.e., the time to react to a stimulus) of a person in the presence of "choices".

The law models human reaction time (also called choice-reaction time) under uncertainty (the presence of choices). The law states that the reaction (decision) time T increases with uncertainty about the judgment or decision to be made.

Hick's Law is directly relevant to the kiosk's route list: the real BEST network has close to five hundred routes, and showing all of them on screen at once would make choosing among them slow and error-prone by Hick-Hyman's Law (reaction time grows with the logarithm of the number of choices).

Application of Hick's Law in the kiosk:

- **Pagination limits the visible choice set:** only eight route rows are shown at a time rather than all ~495 at once, keeping the immediately visible number of alternatives small even though the full catalogue remains one scroll away.
- **Search collapses the effective choice set toward one:** a rider who already knows their route number or stop name can type it on the on-screen keypad, filtering hundreds of candidates down to a handful (or exactly one) almost immediately, which is the direct, interactive answer to Hick-Hyman's Law's prediction that more choices means slower decisions.
- **The Stops screen reuses the same pattern:** a route's stop list is similarly paginated and searchable, so a long route with over a hundred stops does not force the rider to scan them all before finding their own.

By combining pagination with an opt-in search, the kiosk lets a rider who wants to browse do so at a manageable, low-choice pace, while a rider in a hurry who already knows what they want can shortcut Hick-Hyman's Law's reaction-time penalty entirely.

Guidelines in HCI

Shneiderman’s eight golden rules

- **Strive for Consistency:** every screen shares one header bar (title left, language switch top-right), one colour language (teal/blue for a primary action, grey for secondary/Back, green for success/start, orange for destination/end, red-orange for an error), and one button shape (a filled rectangle with a white, centred label) across the route, stops, payment, ticket, and help screens. The on-screen keypad reuses the exact same button style as every other control, rather than a separate widget style.
- **Enable Frequent Users to Use Shortcuts:** the route-number and stop-name search lets a rider who already knows their route code (for example "213") skip straight past browsing or scrolling hundreds of entries -- the same shortcut-for-experts role a type-ahead field plays on a desktop app, adapted to touch-only input.
- **Offer Informative Feedback:** every tap has a visible consequence: buttons change screens immediately, the selected stop turns green (start) or orange (end), the search query box echoes exactly what has been typed with a blinking cursor, and the computed fare is shown before the rider commits to paying.
- **Design Dialogs to Yield Closure:** the flow is a strict, visible sequence -- route, then stops, then payment, then ticket -- each with its own screen and a clear terminal state (a completed, printed-looking ticket screen), not an open-ended form.
- **Offer Simple Error Handling:** booking validation rejects an incomplete or same-start/end booking before money changes hands, and a printer-out-of-paper condition surfaces as a plain-language, translated notice rather than a crash or a silently lost ticket.
- **Permit Easy Reversal of Actions:** a Back button exists on every screen except the initial route list (where there is nothing to go back to), and selecting a different start/end stop or a different payment method is a single tap with no confirmation gauntlet. Opening the search keypad never commits a filter irreversibly -- Clear and further edits remain available until Done is tapped.
- **Support Internal Locus of Control:** the rider drives every transition (tap a route, tap stops, tap pay); the system never auto-advances or times out mid-flow, consistent with a public kiosk where the person, not the machine, should feel in charge of the transaction.
- **Reduce Short-Term Memory Load:** the chosen route's number and name stay printed at the top of the Stops screen so the rider never has to remember what they picked two screens back; the map keeps every real stop visible as a small circle for spatial recognition instead of requiring the rider to recall stop names from memory.

Norman’s seven principles

- **Use both knowledge in the world and knowledge in the head:** the kiosk leverages knowledge in the world by always showing the map, the stop list, and the current selection state on screen, so a rider does not need to hold the whole transaction in their head -- while still letting a rider who already knows their route number use it directly (knowledge in the head) via search.
- **Simplify the structure of tasks:** the booking task is broken into four small, sequential decisions (route, start stop, destination stop, payment method) rather than one large form, so each individual decision stays cognitively simple even though the overall network is large.
- **Make things visible:** route list, map, zoom controls, search, language switch, and help are all on-screen simultaneously within their screen's context, not nested behind a settings icon; nothing needed to operate the kiosk is hidden.
- **Get the mapping right:** the map and the stop list are two views of the same selection state -- tapping a stop's dot on the satellite map and tapping the same stop in the sidebar list do the same thing, so the rider's mental model (“this dot = this

list row") holds throughout.

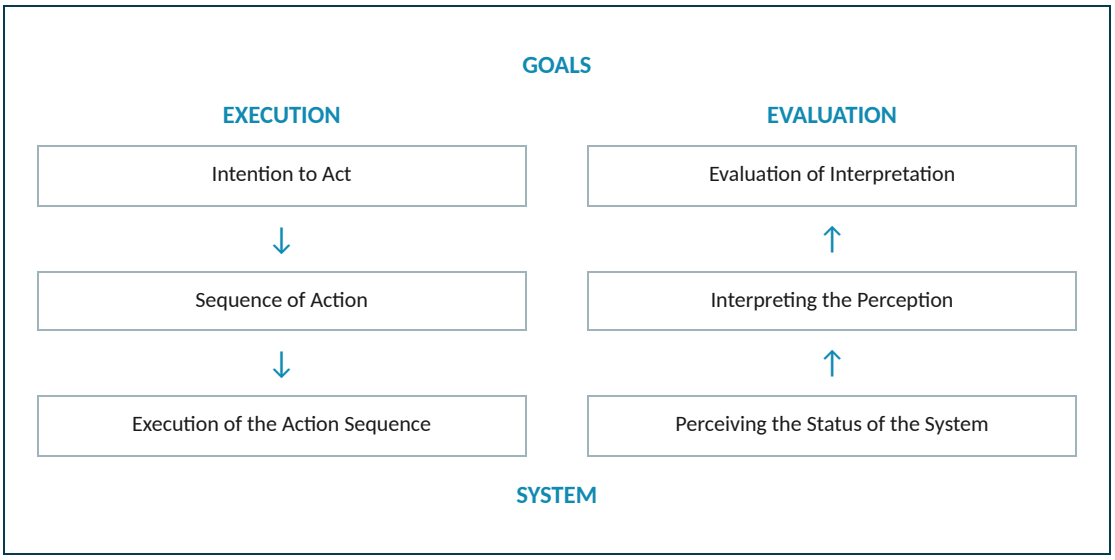
- **Convert constraints into advantages:** the on-screen keypad only offers Latin letters and digits, which is exactly the alphabet route codes and the underlying English stop data use, so there is no way to type a query that could never match anything; the zoom camera is clamped so it can never pan past the edge of the basemap, structurally preventing a lost-in-the-map state.
- **Design for error:** Continue is only reachable once both a start and end stop are chosen and they differ; a printer-paper error is caught before a ticket is falsely promised; and nearly every choice (a stop, a payment method) can be changed with a single further tap rather than requiring the rider to restart.
- **When all else fails, standardize:** colour and button conventions (green = start/success, orange = destination, grey = neutral/secondary/back, teal/blue = primary action) are never reused with a different meaning on a different screen, giving the rider one consistent visual language across the whole application.

Norman's model of interaction

Norman's Model of Interaction consists of seven stages, as follows:

- User establishes a goal
- User formulates an intention
- User decides on an action
- User executes the action
- User perceives the system state
- User interprets the system response
- User evaluates the system with respect to the goal

Another way of depicting Norman's seven-stage Action model, applied to the kiosk's own stop-selection step, is shown below:



Applied concretely: a rider's goal is "reach my destination"; their intention becomes "select this bus stop as my boarding point"; the action sequence is "tap the stop's dot"; execution is the actual tap; the system's response is the dot turning green; the rider perceives that colour change, interprets it as "my start stop is now selected", and evaluates it against their original goal before moving on to pick a destination stop.

Nielsen’s ten heuristics

- **Visibility of system status:** the booking/payment flow always shows which screen the rider is on via the header title; map zoom level and scroll position are reflected live (scroll arrows grey out at the start or end of a list, exactly like a disabled button should).
- **Match between system and the real world:** real BEST route numbers, real stop names, and a real satellite view of Mumbai replace what might otherwise be invented routes on a generic mock map -- the system matches the actual world the rider is navigating, not an abstraction of it.
- **User control and freedom:** Back buttons and a persistent, editable search query (not a one-shot filter) mean a rider can always retreat from or revise a choice; nothing is a one-way door except completing payment.
- **Consistency and standards:** one shared button/box/text vocabulary is used across the whole application; green always means "start/success", orange always means "destination", grey always means "neutral/secondary/back" -- colours are never reused with a different meaning on a different screen.
- **Error prevention:** fixed-size catalogue bounds plus the catalogue-generation tooling’s validation prevent a malformed catalogue from ever reaching the firmware; booking validation prevents a same-stop or incomplete booking from ever reaching the payment screen in the first place, which is stronger than catching the error after the fact.
- **Recognition rather than recall:** the satellite map plus always-visible stop dots let a rider recognise their stop by its real-world location instead of having to recall its exact printed name; the scrollable list is always paired with the same visual map, so either memory style (place or name) works.
- **Flexibility and efficiency of use:** the on-screen search keypad is the direct answer to this heuristic: a novice can browse or scroll the full list, while someone who already knows a route number or stop name can search straight to it -- the same two-speed interaction Nielsen’s rule asks for, without adding a mode switch the novice path has to route around.
- **Aesthetic and minimalist design:** the map never renders all routes’ coloured lines simultaneously -- only the always-present stop dots plus whichever single route is currently selected -- a deliberate choice to keep the screen legible at real-world data scale instead of letting "more real data" become visual noise.
- **Help users recognize, diagnose, and recover from errors:** the printer-empty notice is plain language, translated into the rider’s chosen language, placed right above the action that triggered it -- not an error code or a stack trace.
- **Help and documentation:** a dedicated, one-tap-away Help screen carries the helpline number, an emergency number, and the data/imagery attributions, reachable from the route screen without derailing an in-progress booking.

Multilingual access deserves a specific note here, since it is really Shneiderman and Nielsen's consistency and recognition heuristics applied across languages, not just within one: English, Marathi, Hindi, and Gujarati each get a dedicated, bundled font rather than relying on whatever happens to be installed on the host, specifically because inconsistent rendering (text silently turning into empty boxes) is itself a usability failure -- a rider who cannot read the UI at all has had every other heuristic above revoked for them.

Task Modeling and Analysis

Hierarchical Task Analysis (HTA)

HTA (Hierarchical Task Analysis) is a method used in Human-Computer Interaction (HCI) to represent the hierarchy of tasks and subtasks involved in a system. Below is an HTA for the kiosk's overall "Book a Ticket" task.

Task: Book a Ticket

A rider uses the BEST Mumbai Embedded Ticket Kiosk to select a route, choose boarding and alighting stops, pay, and receive a ticket.

1. Select a route

1.1. View the route list and satellite map.

1.2. Optionally, tap the search field and type a route number on the on-screen keypad.

1.3. Scroll the route list, if needed, using the up/down arrows.

1.4. Tap the desired route row.

1.5. Observe the map recentre and zoom onto the selected route.

2. Select start and destination stops

2.1. View the stop list and the selected route highlighted on the map.

2.2. Optionally, tap the search field and type a stop name.

2.3. Tap the boarding stop, on either the map or the list.

2.4. Observe the boarding stop turn green.

2.5. Tap the destination stop, on either the map or the list.

2.6. Observe the destination stop turn orange.

2.7. Optionally, tap Timetable to check the route’s departure schedule.

2.8. Tap Continue to proceed to payment.

3. Confirm and pay

3.1. Read the displayed route, stops, and computed fare.

3.2. Tap a payment method: Cash, Card, or UPI.

3.3. Observe the chosen method highlighted green.

3.4. Tap "Print ticket" to complete the booking.

3.5. If the printer is out of paper, read the translated notice and retry once resolved.

4. Receive and save the ticket

4.1. View the completed ticket, including the QR code and validity window.

4.2. Optionally, tap "Save PDF" to store the ticket as a file.

4.3. Optionally, tap "Print" to send the ticket to a physical printer.

4.4. Optionally, tap "New ticket" to begin another booking.

Concur Task Tree (CTT)

Here is the Concur Task Tree (CTT) representation for the kiosk’s major feature groups --

Route Selection:

- View the scrollable route list and satellite map
- Scroll the route list up or down
- Search for a route by number
- Select a route
- Pan or zoom the map

Stop Selection:

- View the stop list and the selected route on the map
- Scroll the stop list up or down
- Search for a stop by name
- Select a boarding stop
- Select a destination stop
- De-select a previously chosen stop

Timetable Lookup:

- Open the timetable for the selected route
- View the grid of departure times
- Return to the previous screen

Payment:

- View the route, stops, and computed fare
- Select Cash, Card, or UPI as the payment method
- Confirm and print the ticket
- Read and respond to a printer-paper error notice
- Return to the Stops screen

Ticket:

- View the completed ticket and QR code
- Save the ticket as a PDF
- Print the ticket
- Start a new ticket

Language Switching:

- View the current language
- Switch to the next language in the cycle (English → Marathi → Hindi → Gujarati)

Help:

- Open the Help screen
- View the helpline and emergency numbers
- View data and imagery attributions

- Return to the Route screen

Evaluation Techniques

Cognitive Walkthrough

Cognitive walkthrough is a usability method that focuses on evaluating a design (existing or proposed) for ease of learning, particularly through explorations by the user. Cognitive Walkthroughs (CWT) are a Usability Inspection Method (UIM) that focuses on evaluating a design's navigation, with ease of learning by self-exploration by the user as the basis.

CWT Questions:

- Will the user try to achieve the right effect?
- Will the user notice that the correct action is available?
- Will the user associate the correct action with the effect that the user is trying to achieve?
- If the correct action is performed, will the user see that progress is being made towards the solution of the task?

Cognitive Walk Through has two phases:

- **Preparation phase:** building a prototype (paper, mock-up, or screen-based) with description; it need not be perfect or complete in all respects. Making a list of selected tasks to "walk through" the interface along with the user, each with a well-defined sequence of goals and sub-goals and the written actions used to complete it. A clear understanding of the user, their background, level of expertise in the domain, and prior experience of using similar software.
- **Evaluation phase:** conducting the walkthrough session, recording the observations, analysis, inferences, and recommendations to the interface team.

Feature Number from above list	Did the user try to achieve the end goal or did they give up at the start itself?	Did the user notice that the correct action choices are available?	Did the user confidently know that the choice being made by him/her is the right one?	Did the user understand the feedback after every action?	Did the user Complete the Task With satisfaction - Yes/PARTLY/No	Comments / Alternative suggestions/ solutions
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						

Analysis & Inference - Evaluators Rating Sheet

Action in Sequence	System mismatch question list	Potential Problem & Design solution	% Mismatch to ideal situation
Route Selection (Map and Searchable List)			
Stop Selection with Map and List Redundancy			
On-Screen Keypad Search for Routes and Stops			
Four-Language Multilingual UI			
Offline Satellite Basemap with Discrete Zoom and Pan			
Bus Timetable Lookup			
Fare Calculation and Payment Method Selection			
Scannable QR-Coded Digital Ticket			
Save/Print Ticket as PDF			
Printer-Paper Error Handling			
Helpline and Dedicated Help Screen			

Heuristic Evaluation

A heuristic is a simple principle or "rule of thumb". A heuristic evaluation is one where an evaluator or group of evaluators walks through the design of an interface and decides whether or not it complies with these "rules of thumb". It is a process where experts use rules of thumb to measure the usability of user interfaces in independent walkthroughs and report issues. Evaluators use established heuristics (for example, Nielsen and Molich's) and reveal insights that can help design teams enhance product usability from early in development.

10 Usability Heuristics

- Visibility of system status
- Match between system and the real world
- User control and freedom
- Consistency and standards
- Error prevention
- Recognition rather than recall
- Flexibility and efficiency of use
- Aesthetic and minimalist design
- Help users recognize, diagnose, and recover from errors
- Help and documentation

Action in Sequence	Evaluator 1 - Feedback	Evaluator 1 - Severity	Evaluator 2 - Feedback	Evaluator 2 - Severity	Evaluator 3 - Feedback	Evaluator 3 - Severity
Route Selection (Map and Searchable List)						

Action in Sequence	Evaluator 1 - Feedback	Evaluator 1 - Severity	Evaluator 2 - Feedback	Evaluator 2 - Severity	Evaluator 3 - Feedback	Evaluator 3 - Severity
Stop Selection with Map and List Redundancy						
On-Screen Keypad Search for Routes and Stops						
Four-Language Multilingual UI						
Offline Satellite Basemap with Discrete Zoom and Pan						
Bus Timetable Lookup						
Fare Calculation and Payment Method Selection						
Scannable QR-Coded Digital Ticket						
Save/Print Ticket as PDF						
Printer-Paper Error Handling						
Helpline and Dedicated Help Screen						

Thus, Heuristic evaluation (Nielsen and Molich, 1990; Nielsen 1994) is a usability engineering method for finding the usability problems in a user interface design so that they can be attended to as part of an iterative design process.

Model Based Evaluation

Model-based evaluation is using a model of how a human would use a proposed system to obtain predicted usability measures by calculation or simulation. These predictions can replace or supplement empirical measurements obtained by user testing. In addition, the content of the model itself conveys useful information about the relationship between the user's task and the system design. Model-based evaluation can be best viewed as an alternative way to implement an iterative process for developing a usable system. The Keystroke-Level Model and CMN(GOMS) estimates earlier in this report (for route/stop selection and for payment) are themselves examples of this kind of evaluation, contrasted here with the standard, user-testing-driven usability process described below.

Evaluation with 5 goals of Usability Engineering

Goal of Usability Engineering	Rating out of 10
Effective to use - Functional	
Efficient to use - Efficient	
Error free in use - Safe	
Easy to use - Friendly	
Enjoyable in use - Pleasurable	
Total	

Implementation

Technology Used

ISO C17

The kiosk's firmware core -- route/stop catalogue, booking rules, localization strings, and map projection math -- is written in portable, bounded ISO C17. Keeping this layer free of any graphics dependency is what lets it be unit tested the same way any other pure-logic module would be, and is what makes an eventual port to real embedded hardware realistic rather than purely aspirational.

SDL2 + SDL2_ttf (HarfBuzz-enabled)

SDL2 provides the graphical framebuffer and mouse-as-touch input used to emulate the eventual display during development. SDL2_ttf, built with HarfBuzz support, is responsible for complex-script text shaping -- correctly rendering conjuncts, reph, and matra reordering in Devanagari and Gujarati, not just simple Latin text layout.

Bundled Noto fonts

Noto Sans, Noto Sans Devanagari, and Noto Sans Gujarati (all OFL-1.1 licensed) are bundled directly with the application rather than relying on whatever happens to be installed on the host or target device, guaranteeing correct, portable rendering for English, Marathi, Hindi, and Gujarati regardless of deployment environment.

Offline satellite basemap and real route data

The basemap is a Sentinel-2 cloudless satellite mosaic (EOX, CC BY-NC-SA 4.0), fetched once and bundled so the running kiosk itself never touches the network. Route and stop data is the real BEST catalogue, imported once from the community-maintained croyla/mumbai-gtfs feed (MIT-0 licensed) rather than hand-authored mock data.

JSON configuration and Python tooling

Editable route and map-bounds configuration is kept as committed, human-reviewable JSON, generated into fixed-size C arrays/structs at build time by a network-free, standard-library-only Python script. This means the firmware path needs no parser or dynamic allocation at runtime regardless of how large the real dataset gets, while the source configuration remains easy for a maintainer to inspect or edit.

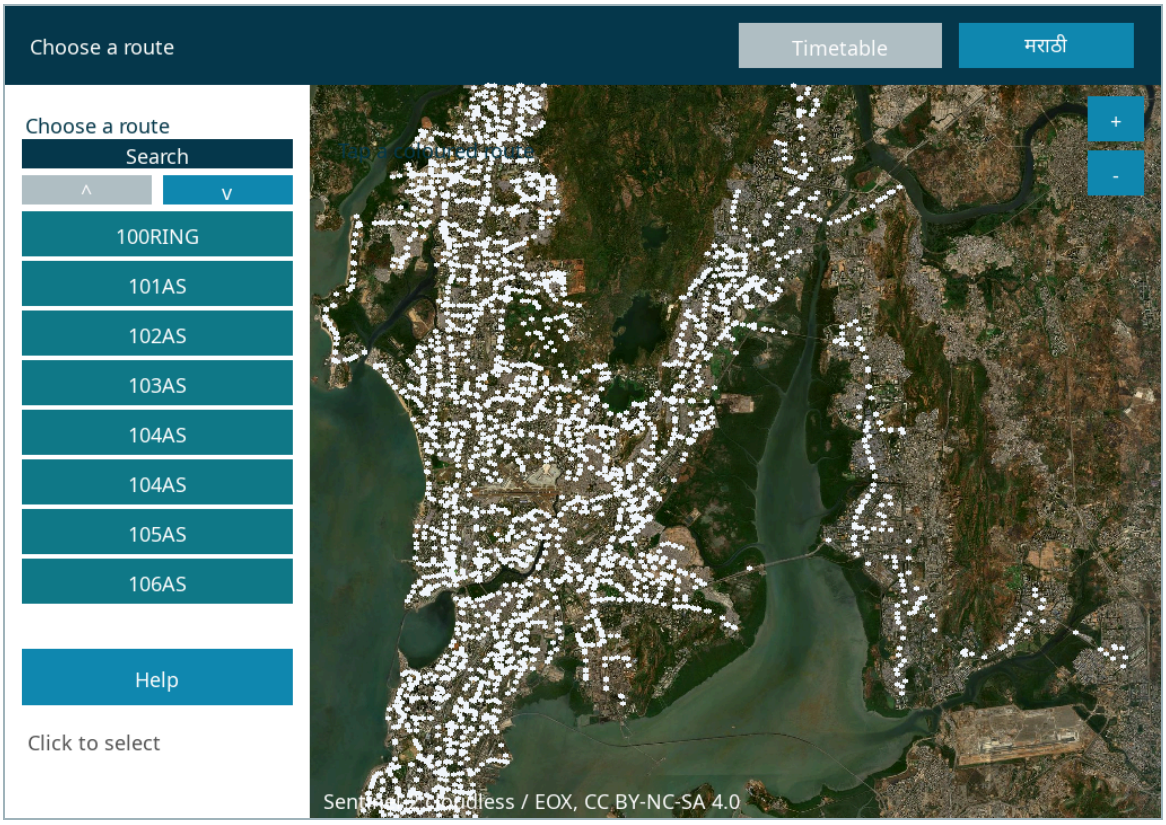
CMake, Ninja, and CTest

The project builds reproducibly with CMake and Ninja, and every SDL-free core module (catalogue, booking, localization, map projection, search, and timetable lookup) has its own CTest-registered unit test executable, so correctness of the business logic can be verified independently of the graphical front end.

Future board target: QEMU mps2-an385

SDL2 is explicitly the development display emulator, not the final target. A QEMU mps2-an385 Cortex-M3 board is the intended next step once a specific display/touch hardware-abstraction layer is selected, since no universal touchscreen/framebuffer device exists for that board today; the current dataset and memory budget would need re-evaluating for a genuinely constrained microcontroller before that port.

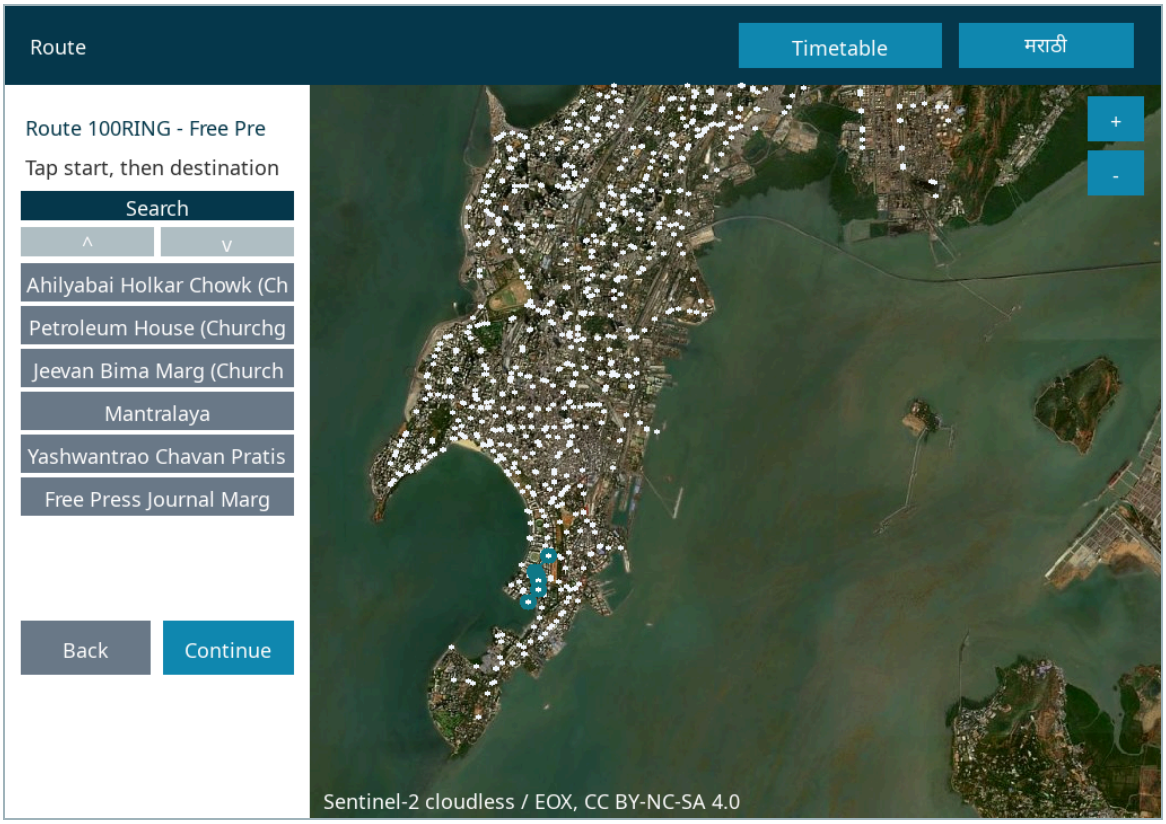
Results



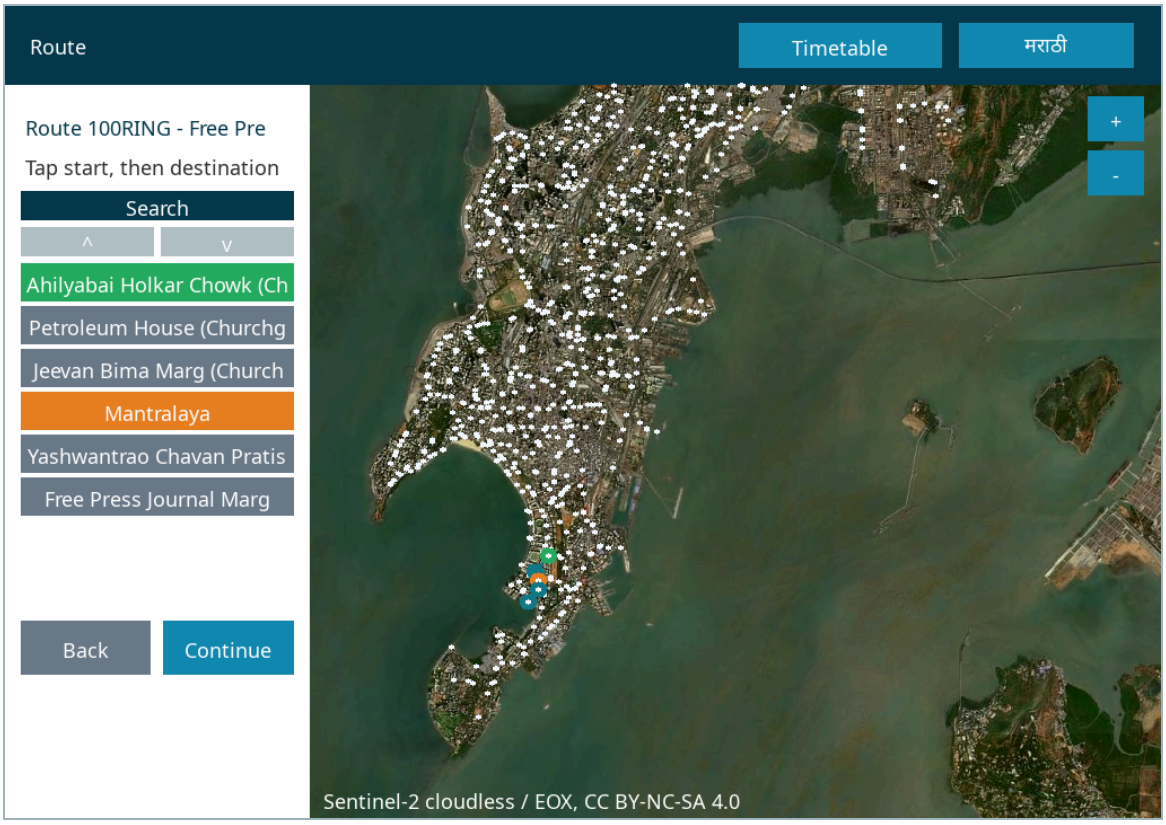
1. Route Selection screen -- the satellite map of Mumbai with every real BEST stop drawn as a dot, alongside a scrollable, colour-coded route list and a Help button.



2. On-screen keypad search, opened from the Route screen -- Latin letters/digits only, with Clear, backspace, and Done controls.



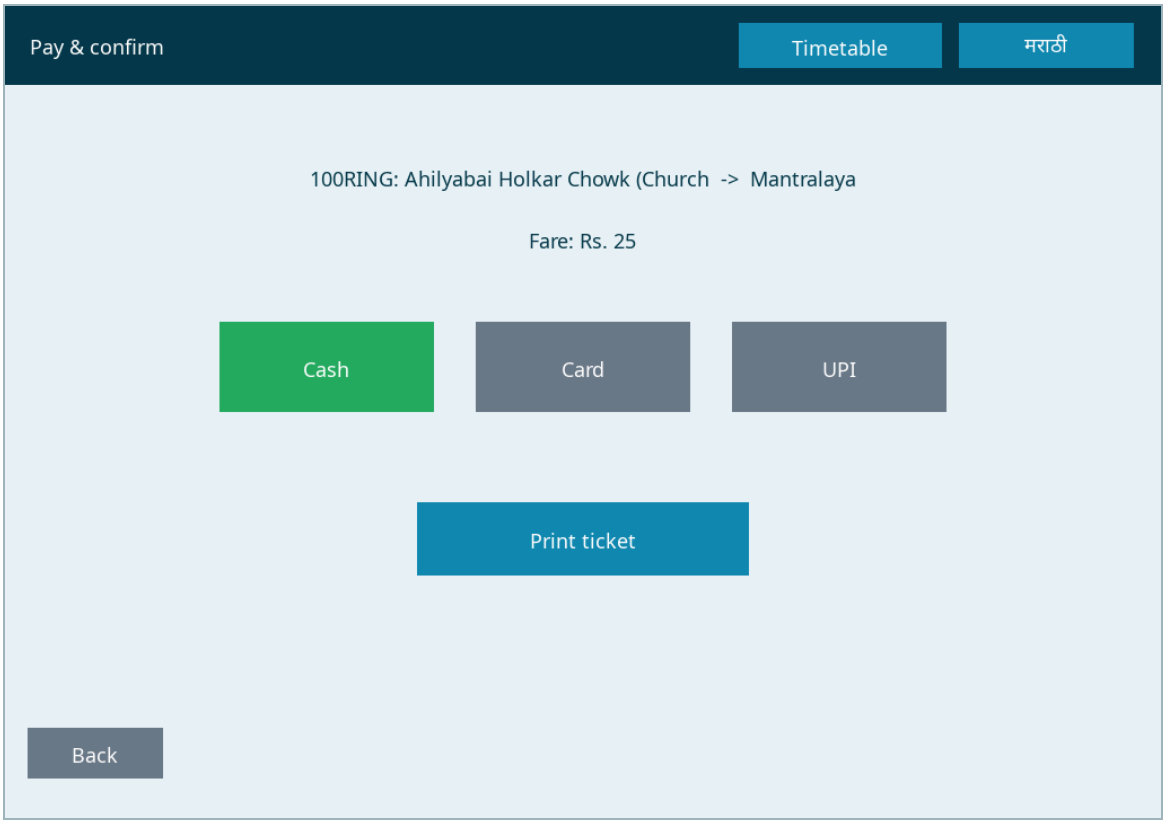
3. Stop Selection screen for route 100RING -- the chosen route's stops, listed and highlighted on the map, with Back and Continue actions.



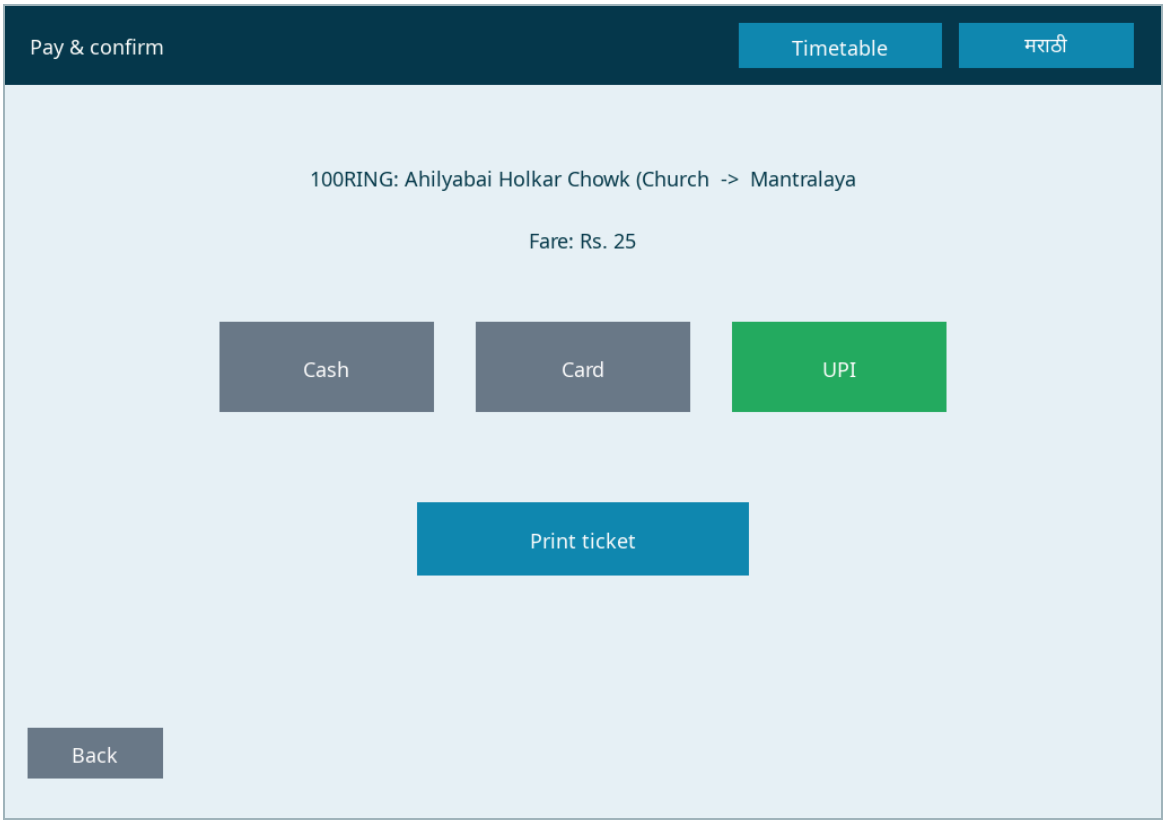
4. A boarding stop (green) and a destination stop (orange) selected -- shown consistently in both the sidebar list and on the map.



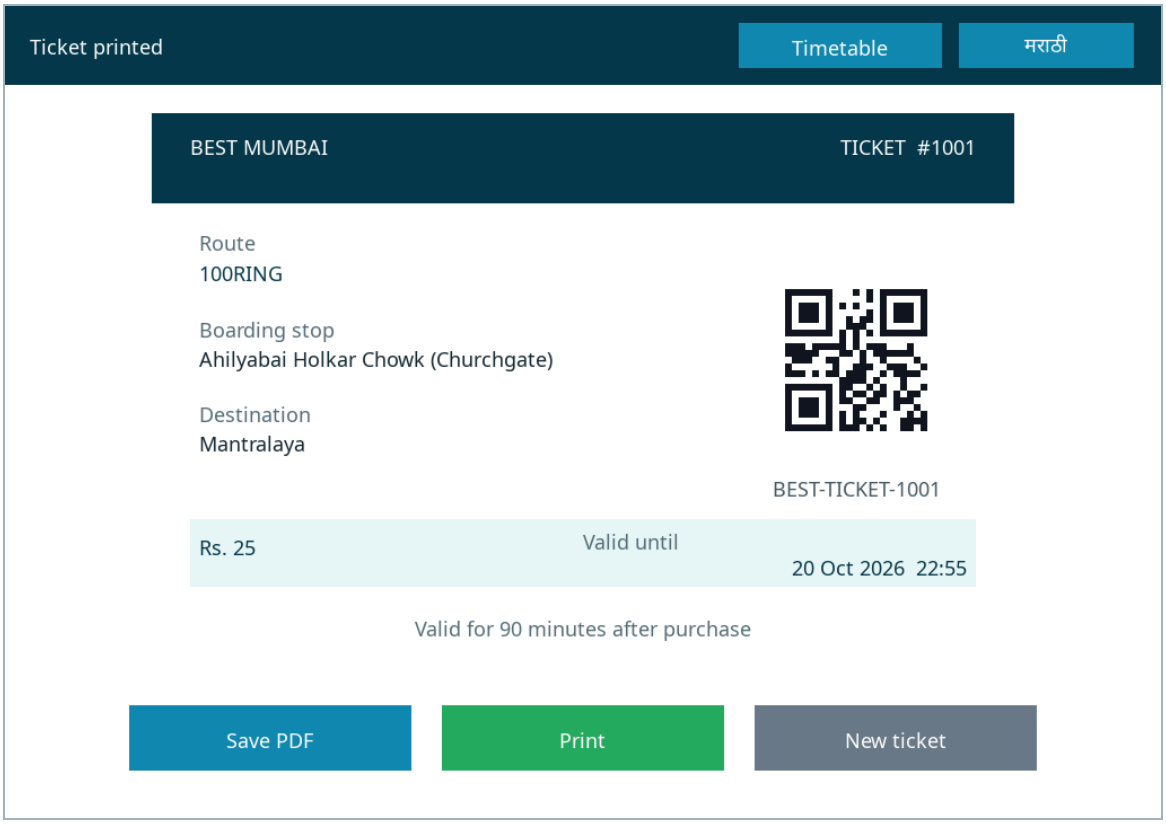
5. Timetable screen for the selected route -- a fixed grid of real departure times needing no scrolling or search.



6. Payment screen -- route, stops, and computed fare shown together, with Cash/Card/UPI options and a Print ticket action.



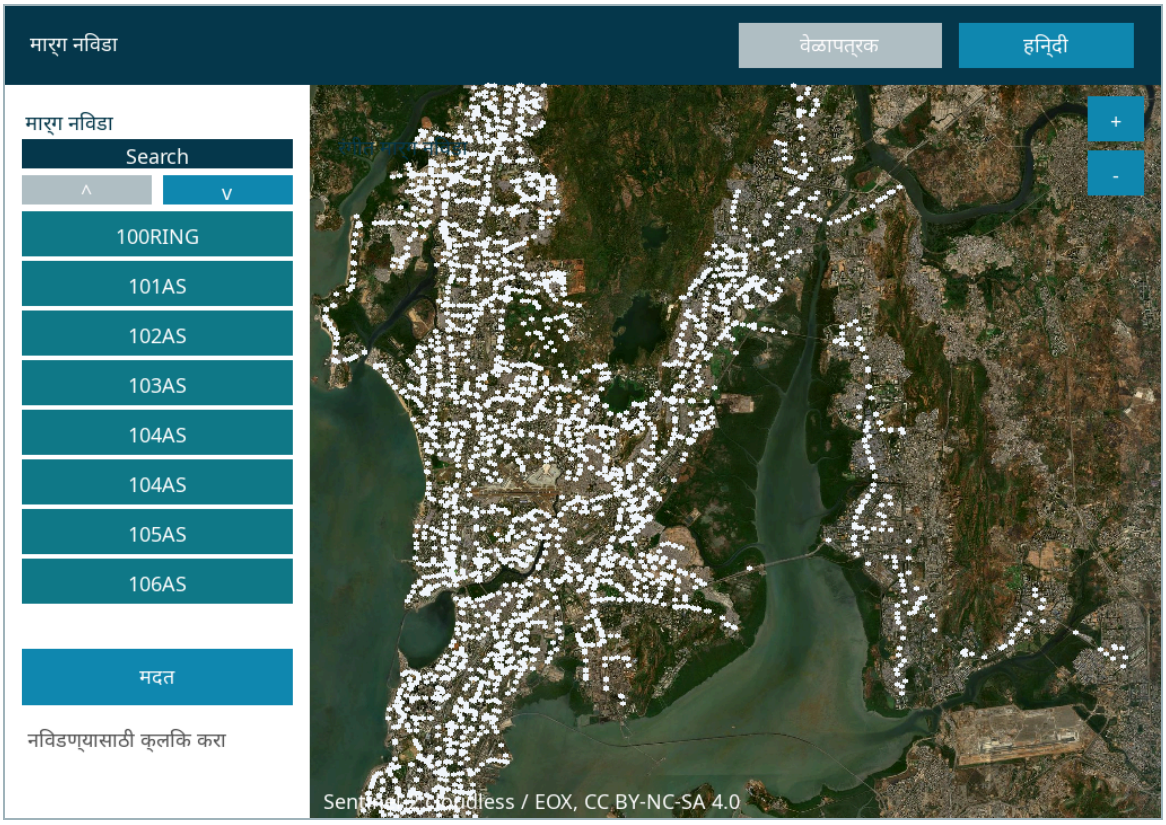
7. UPI selected as the payment method, highlighted green for an unambiguous choice.



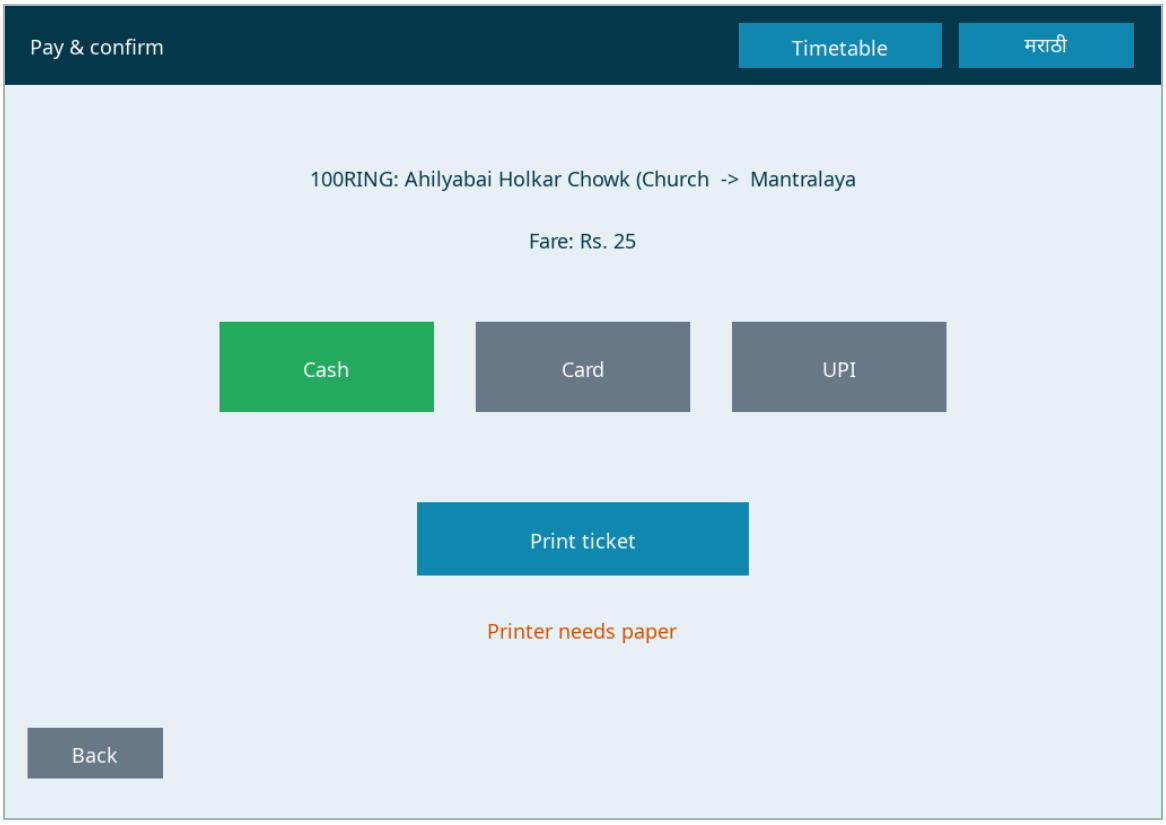
8. Completed ticket -- route, boarding stop, destination, fare, validity window, and a scannable QR code, with Save PDF/Print/New ticket actions.



9. Help screen -- helpline number, emergency number, and data/imagery attributions, one tap away from the Route screen.



10. The Route screen in Marathi -- fixed UI text, the route list, and the Help button all rendered in the switched language.



11. Printer-paper error handling -- a plain-language, translated notice ("Printer needs paper") surfaced on the Payment screen instead of a silently failed booking.

Future Scope

- **Port to a real embedded target:** move the display/touch path from the SDL2 desktop emulator to a genuine constrained-MCU board (building on the QEMU mps2-an385 groundwork already noted in the project's architecture), re-evaluating the route/stop memory budget for real hardware constraints.
- **Real GPIO printer integration:** replace the simulated paper-stock sensor with a real GPIO-driven thermal printer, keeping the same booking-validation and error-notice logic already in place.
- **GTFS-realtime integration:** consume a live GTFS-realtime feed for actual delays and vehicle positions, rather than only a static, sampled timetable.
- **Contactless and NFC payment:** extend the Cash/Card/UPI payment options with tap-to-pay NFC card or transit-card support, common on comparable public-transit kiosks.
- **Backend-verified QR tickets:** pair the kiosk with a ticketing backend so the QR code printed on each ticket can be validated in real time by a conductor's or inspector's scanner, rather than only serving as a prototype reference.
- **Accessibility enhancements:** add audio or voice cues and a larger-text "accessibility mode" for visually impaired or low-literacy riders, extending the project's existing multilingual-access philosophy to other axes of accessibility.
- **A fifth (or further) language:** the localization architecture already supports adding a new language by adding a string array and a bundled font, so further regional languages could be added with no change to the booking logic itself.
- **Operational analytics for BEST:** aggregate anonymised booking data (without personally identifying a rider) into a dashboard for BEST operations staff, surfacing route demand and peak-time patterns.

These future scope points aim to carry the project from a desktop interaction prototype toward a deployable embedded kiosk, while preserving the multilingual accessibility and structural error handling already central to its design.

Conclusion

In conclusion, the BEST Mumbai Embedded Ticket Kiosk is a touch-only, multilingual ticket-booking kiosk prototype for BEST Mumbai's real bus network. The application lets a rider independently pick a route, pick boarding and destination stops via a map or a searchable list, choose a payment method, and receive a scannable, time-limited QR ticket that can be saved or printed as a PDF, all without needing a conductor or a shared spoken language.

The project's main objectives include proving a touch-only, click-as-touch interaction model, delivering multilingual access as a first-class requirement rather than an afterthought, and working against the real BEST route/stop catalogue and basemap rather than a simplified mock-up. BEST Mumbai Embedded Ticket Kiosk addresses the limitations of a purely conductor-mediated, paper-ticket process by making the transaction visible and checkable before payment, structurally validating a booking before money changes hands, and surfacing failure states such as a printer-paper-empty condition as plain-language, translated notices rather than silent failures.

The future scope of the project includes porting the display/touch path to real embedded hardware, replacing the simulated printer sensor with a real GPIO driver, consuming live GTFS-realtime data, and adding backend-verified QR ticket scanning and contactless payment. With its SDL-free, unit-tested business logic and its real-scale, real-data design, the kiosk aims to demonstrate a credible first step toward a genuinely deployable, accessible self-service ticketing kiosk for BEST Mumbai's riders.